Open AI Cubes: a concurrent, parallel and distributed CNN with sockets in Java, Go and Python

Espinoza Benavente Hector
Inga Chávez Jorly
Villanueva Alzamora Farid

Universidad Nacional de Ingeniería – Facultad de Ciencias

CC4P1 – Concurrent and Distributed Programming
Midterm Exam – 2026-II

Contents

1 Introduction **2**

2 Theoretical framework **2**
2.1 Convolutional neural networks . . . 2
2.2 Distributed systems and socket communication . . . 2
2.3 Concurrency and synchronization . . . 3
2.4 Performance . . . 3

3 Methodology and implementation **3**
3.1 System architecture . . . 3
3.2 CNN model and micro chunks . . . 4
3.3 Parallelism inside a node and distribution across nodes . . . 6
3.4 Communication protocol . . . 6
3.5 Program development by language . . . 7
3.6 Deployment . . . 8
3.7 Experimental design . . . 9

4 Results and discussion **10**
4.1 Time matrix (12 cases) . . . 10
4.2 Scalability: speedup and communication cost . . . 11
4.3 Load test . . . 12
4.4 General discussion . . . 13

5 Conclusions **13**

6 Source code appendix **14**

7 Documentation appendix **14**

1 Introduction

Open AI Cubes is an architectural concept for modular artificial intelligence systems in which each "cube" provides one capability, and the cubes cooperate in parallel from different nodes to process a convolutional neural network (CNN). In this work we design and implement a working version of that idea focused on image processing: the system trains and serves a CNN that recognizes four CIFAR-10 classes (airplane, automobile, cat and ship).

The system consists of a **central server written in Java**, deployed on a **Google Cloud** virtual machine, and **three compute nodes**, one per team member and each one in a different language: **Java, Go and Python**, running on three physical laptops. Users interact through a chat-style desktop interface: they send an image and receive the recognized class, the confidence and the node that performed the computation.

As required by the assignment, everything is built only with the standard SDK of each language: TCP sockets, threads and synchronization structures, with no external libraries, no WebSocket, no Socket.IO and no middleware. Communication uses a custom binary protocol (OC/1). The CNN is split into convolution, pooling and multilayer perceptron *micro chunks*, which each node computes in parallel and which the server distributes among the nodes.

The goals of this work are:

- Implement the CNN with concurrent, parallel and distributed micro chunks in three languages.
- Integrate the image processing, *backend* and *frontend* modules into a distributed system with a custom protocol.
- Evaluate execution times in the sequential, parallel, distributed and parallel-distributed scenarios for cases 1, 2 and 3 of the assignment.
- Load the system with hundreds of clients using a random *delay* to find its limits.

2 Theoretical framework

2.1 Convolutional neural networks

A CNN is a deep learning architecture designed for grid-structured data such as images. It has a convolutional part, which extracts features, and a multilayer perceptron, which classifies the input from those features.

2.1.1 Convolutions, ReLU and pooling

A convolution slides small filters (here 3×3) over the image. Each filter produces an activation map that responds to a local pattern, such as an edge or a texture:

$$\text{out}[f][y][x] = b_f + \sum_{c} \sum_{k_y} \sum_{k_x} W_{f,c,k_y,k_x} \, \text{in}[c][y + k_y - 1][x + k_x - 1].$$

The ReLU function, $\max(0, z)$, introduces non-linearity, and 2×2 *max pooling* halves each spatial dimension while keeping the maximum value of each window.

2.1.2 Multilayer perceptron and softmax

The final map is flattened into a vector that goes through fully connected layers. The last one produces one value per class, which the softmax function turns into probabilities. Training minimizes the cross-entropy loss with stochastic gradient descent (SGD) with momentum.

2.2 Distributed systems and socket communication

A distributed system is made of autonomous processes that communicate over a network to work on a common task. Communication is done with *sockets*: a socket is one endpoint of a bidirectional channel between two processes. TCP is used because it guarantees reliable, in-order delivery, and the protocol relies on that: for example, the network weights always reach a node before the task that uses them.

2.3 Concurrency and synchronization

When several threads access shared data without coordination, a *race condition* occurs: the result depends on the non-deterministic order in which the operating system interleaves the threads. The tools used to avoid it are:

- **Monitors** (`synchronized`): mutual exclusion over a shared resource.
- `volatile` **variables and atomics** (`AtomicInteger`, `AtomicLong`): visibility across threads and lock-free counters.
- **Producer–consumer** with blocking queues (`BlockingQueue`).
- `Future` **and** `CompletableFuture`: asynchronous results.
- **Barriers**: a group of tasks waits until all of them finish before continuing.

2.4 Performance

Evaluating a concurrent system is not only about whether it works: it is also about measuring how it responds when the workload and the resources grow. The metrics used are:

- **Latency**: time between a client sending a query and receiving the answer.
- **Throughput**: queries or images processed per second.
- **Speedup**: $S = T_{\text{sequential}}/T$, or its throughput equivalent.
- **Parallelization and aggregation cost**: the time that is not useful computation (sending data, waiting and combining results). It is what keeps S from growing linearly.

3 Methodology and implementation

3.1 System architecture

Figure 1 shows the architecture. Clients and nodes always open the connection to the server: clients on port 5000 and nodes on port 6000. Therefore the server does not need to know the node addresses in advance, and the laptops can be on different networks, connected over the internet.

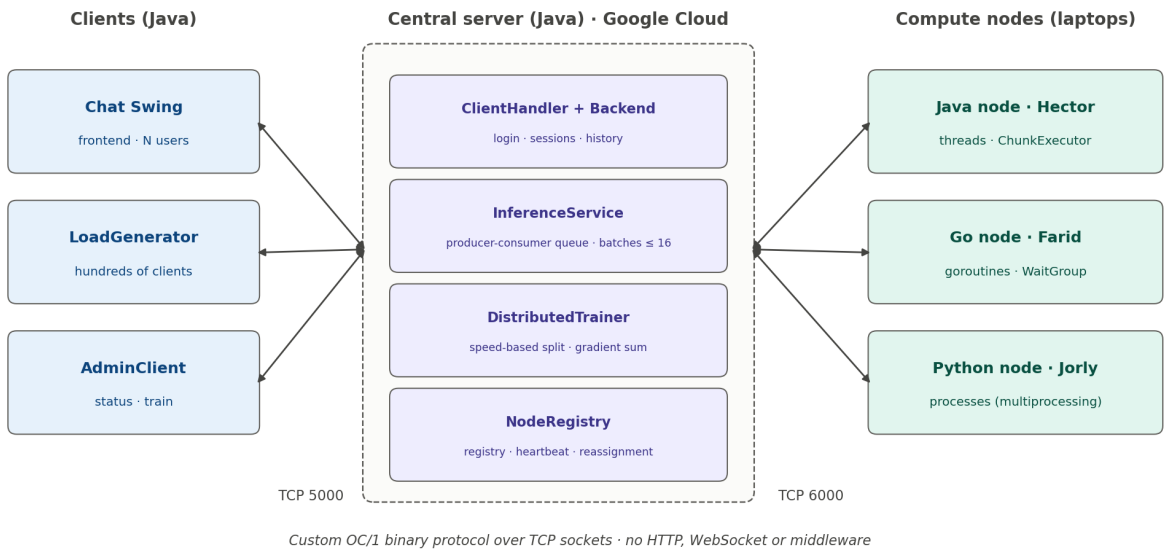


Figure 1: Open AI Cubes architecture: clients, central server and compute nodes.

3.1.1 Modules: images, backend, frontend and integration

- **Image processing:** the CNN, implemented in the three languages, and the `ImagePrep` module, which resizes any photo (JPG, PNG, BMP or GIF) to 32×32.
- **Backend:** the server answers client queries with sessions ("Usuario Validado" and `idClientChat`), a per-user history and an inference queue.
- **Frontend:** a Swing desktop chat (Figure 2). It can send a photo, a whole folder, test images, a batch of 200 images and a load test.

- **Integration:** all modules communicate through the OC/1 protocol (Section 3.4).

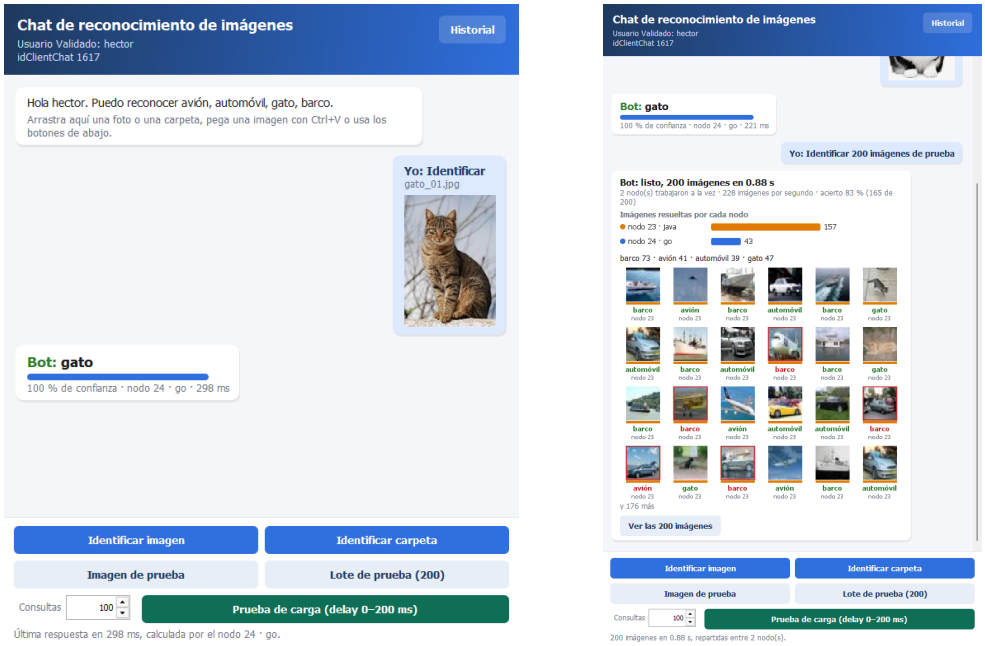


Figure 2: Chat connected to the server on Google Cloud. Left: a photo recognized as a cat with 100 % confidence by the Go node. Right: a batch of 200 test images split among the nodes, with the number of images each one solved.

3.1.2 Code components

Component	Responsibility
`cubes.cnn`	CNN, micro chunks (`ConvMicroChunk`, `PoolMicroChunk`, `MlpMicroChunk`), `ChunkExecutor`, dataset and SGD.
`cubes.net`	Frames and messages of the OC/1 protocol.
`cubes.server`	`CentralServer`, `Backend`, `ClientHandler`, `InferenceService`, `DistributedTrainer` and `NodeRegistry`.
`cubes.node`	Java compute node.
`cubes.client`	Swing chat, load generator (`LoadGenerator`) and admin console.
`go/`	Go compute node (standard library only).
`python/node.py`	Python compute node (standard library only, no numpy).

Table 1: Main components of the implementation.

3.2 CNN model and micro chunks

3.2.1 Dataset

Four CIFAR-10 classes are used: airplane, automobile, cat and ship, with 20 000 training images and 4 000 test images (1 000 per class) of 32×32 RGB pixels. Each record takes 1 label byte plus 3 072 pixel bytes, and the values are normalized to $[-1, 1]$.

3.2.2 Network architecture

Layer	Input	Output	Detail
conv1 + ReLU	$3 \times 32 \times 32$	$16 \times 32 \times 32$	3×3 filter, padding 1
pool1	$16 \times 32 \times 32$	$16 \times 16 \times 16$	2×2 max
conv2 + ReLU	$16 \times 16 \times 16$	$32 \times 16 \times 16$	3×3 filter, padding 1
pool2	$32 \times 16 \times 16$	$32 \times 8 \times 8$	2×2 max
fc1 + ReLU	2 048	128	fully connected
fc2 + softmax	128	4	one output per class

Table 2: CNN layers (267 876 weights in total).

The delivered model correctly classifies 3 292 of the 4 000 test images (82.3 %).

3.2.3 Micro chunks and per-layer barrier

A micro chunk is an independent range of work inside a layer: a group of filters of a convolution, of channels of a *pooling* layer or of neurons of a dense layer. Inside each node, `ChunkExecutor` splits the full range among H threads and waits for all of them to finish (Figure 3). That wait is the barrier between layers, because the next layer needs the complete result of the previous one.

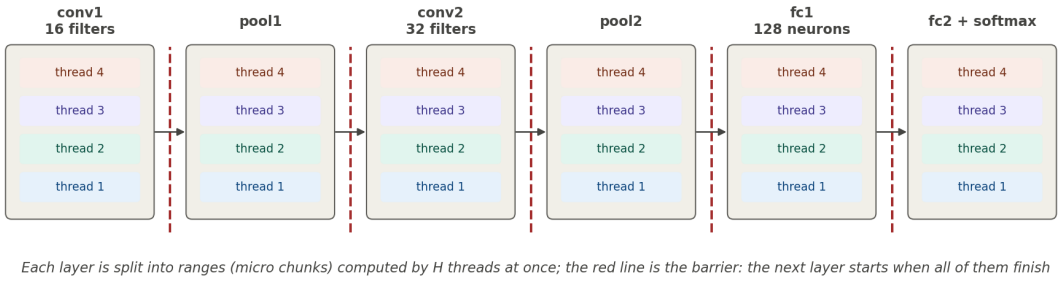


Figure 3: Micro chunks: each layer is split among H threads, with a barrier before the next layer.

```java
public void forRange(int n, RangeTask task) {
    int parts = Math.min(threads, n);
    List<Future<?>> futures = new ArrayList<Future<?>>(parts);
    for (int p = 0; p < parts; p++) {
        int from = n * p / parts, to = n * (p + 1) / parts;   // micro chunk [from, to)
        futures.add(pool.submit(() -> task.run(from, to)));
    }
    for (Future<?> f : futures) {
        f.get();                       // barrier: wait for every micro chunk
    }
}
```

In Go the same barrier is a `sync.WaitGroup` over goroutines. In Python processes are used (`multiprocessing`), because the GIL prevents parallel computation with threads.

3.2.4 Cross-language verification

For the nodes to be interchangeable, the three languages must compute exactly the same thing. The admin console runs a cross-check (`crosscheck`): every node computes the same batch and the server compares the loss, every gradient and the probabilities against a Java reference. During the measurement session, all nine nodes of the three laptops passed the check:

```
Referencia Java local: perdida=13.629654 aciertos=0
nodo 10 [java] DESKTOP-FMBHKTR: perdida=13.629654 err_grad=0.000 err_prob=0.00e+00 -> OK
nodo 11 [go] DESKTOP-FMBHKTR: perdida=13.629654 err_grad=0.000 err_prob=0.00e+00 -> OK
nodo 12 [python] DESKTOP-FMBHKTR: perdida=13.629650 err_grad=0.002 err_prob=6.26e-07 -> OK
...  (all 9 nodes of the 3 laptops: OK)
```

3.3 Parallelism inside a node and distribution across nodes

	Parallel (inside a node)	**Distributed (across nodes)**
What is split	The micro chunks of each layer	The images of each batch
Among whom	H threads, goroutines or processes	The connected nodes, one per machine
Coordination	Per-layer barrier	The server waits for the gradients of every node
Combination	Each thread writes its own range (no races)	The server adds the gradients and applies SGD
Split policy	Equal parts	Proportional to the measured speed of each node

Table 3: What is parallelized and what is distributed.

In distributed training, each batch of 64 images is split among the nodes according to their speed. Each node returns the *sum* of the gradients of its share, and the server adds them in a single thread before updating the weights. In inference, client queries enter a producer–consumer queue and each node takes batches of up to 16 images; a faster node comes back sooner for more work, so the load balances itself.

3.4 Communication protocol

3.4.1 Frame format and messages

Every frame has a 12-byte *big-endian* header: the characters `'O' 'C'`, the version, the message type, a request identifier and the payload length. The response repeats the identifier, which allows several requests in flight over the same connection.

Channel	**Message**	**Purpose**
Node ↔ server	`REGISTER / REGISTER_OK`	The node reports language, cores, threads and machine name.
(port 6000)	`HEARTBEAT`	Every 2 s; with no news for 10 s the node is considered down.
	`SET_WEIGHTS`	Model weights (slot 0 = chat, slot 1 = training).
	`TRAIN_TASK / GRADIENTS`	A share of a batch and the sum of its gradients.
	`INFER_TASK / INFER_RESULT`	Images to classify and their probabilities.
Client ↔ server	`LOGIN / LOGIN_OK`	Session: `idClientChat` and class names.
(port 5000)	`IDENTIFY / RESULT`	One image and its class, confidence, node and time.
	`IDENTIFY_BATCH / RESULT_ITEM`	Many images in a single request.
	`TRAIN_REQ / TRAIN_DONE`	Distributed training from the admin console.
	`STATUS_REQ / STATUS`	Status of nodes, queue and counters.

Table 4: Main messages of the OC/1 protocol.

3.4.2 Sequence diagrams

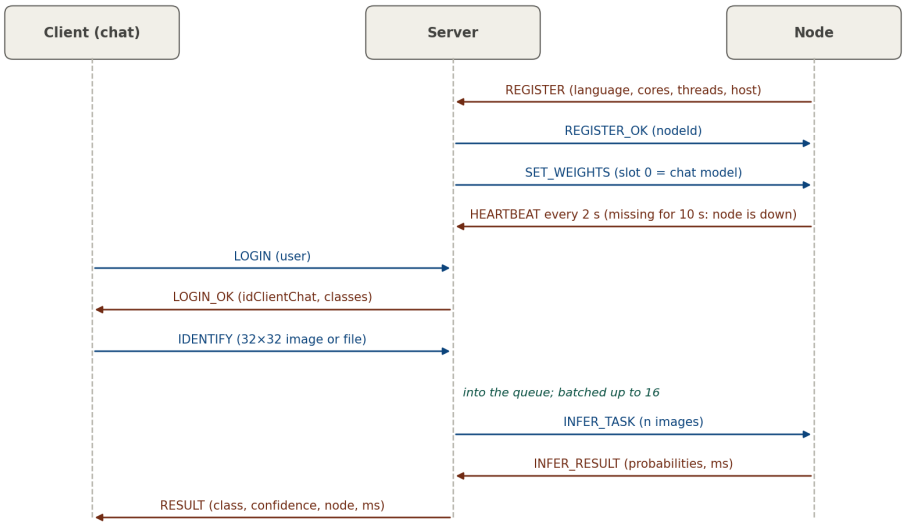


Figure 4: Node registration and an "Identify" query from the chat.

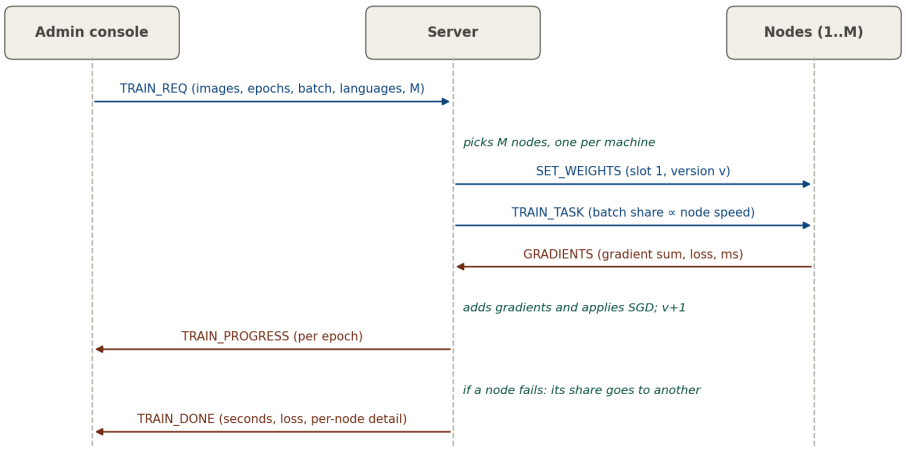


Figure 5: Distributed training of one batch.

3.4.3 Fault tolerance

A node is considered down if its socket closes or if it sends nothing for 10 s (*heartbeat*). Its pending work is reassigned to the fastest live node and the operation still completes. If no node is left, the server answers the chat queries by itself. During the measurements, over home networks, the *heartbeat* detected an unresponsive node three times, and in all three cases its images were reassigned without interrupting training.

3.5 Program development by language

3.5.1 Server, clients and node in Java (Case 1)

The server, the clients and one node are written in Java (LP1 = LP2) and compiled with `-release 8`, so they run on Java 8 or later. The server uses one thread per connection, a *pool* to send tasks to the nodes and `CompletableFuture` to wait for the answers by identifier.

3.5.2 Python node (Case 2)

The Python node (LP2 ≠ LP1) uses only the standard library. To be fast without numpy, it builds the patches of each convolution with index tables and computes the dot products in C through `math` functions. Parallelism comes from processes that live for the whole lifetime of the node.

3.5.3 Go node (Case 3)

The Go node (LP3) splits the micro chunks among goroutines and protects socket writes with a `sync.Mutex`. With the three languages, the team covers Case 3 of the assignment: an additional node in another language.

3.5.4 Concurrency concepts in the code

Concept	Where it is applied
Monitors (`synchronized`)	Socket writes in `ClientHandler`, `ModelStore.publish`, `DistributedTrainer.run`, inference `servingLock`.
`volatile`	Published model in `ModelStore`: every thread sees the new version without locking.
Atomics	`Backend` counters, weight versions and load generator metrics.
Producer–consumer	`InferenceService`: bounded `LinkedBlockingDeque` (rejects instead of crashing).
`Future` / `CompletableFuture`	Node answers by request identifier and training tasks.
Barriers	`ChunkExecutor` (per layer) and `CountDownLatch` so the simulated clients start at the same time.
Race-free splitting	Each node computes its gradients separately; a single server thread adds them.

Table 5: Course concurrency concepts and where they appear.

3.6 Deployment

3.6.1 Laptop cluster

The cluster is made of the three physical laptops of the team, each with its node (Table 6). For the case matrix, each laptop also kept one node of each language on standby: this way the server could form, in each test, a single-language cluster (for example, "all Java" on three machines) by picking one node per machine.

	Laptop 1 (Hector)	Laptop 2 (Farid)	Laptop 3 (Jorly)
Node	Java	Go	Python
Processor	Intel Core i5-13420H	Intel Core i7-1255U	AMD Ryzen 5 5500U
Cores / threads	8 / 12	10 / 12	6 / 12
RAM	32 GB	16 GB	16 GB
Operating system	Windows 11 Pro	Windows 11 Pro	Windows, 64-bit
Java	24.0.2	22.0.1	22.0.2

Table 6: Cluster laptops. Each one connected from a different home network.

3.6.2 Server on Google Cloud

The central server runs on a Google Cloud virtual machine (Compute Engine): an `e2-micro` instance (2 shared vCPUs and 1 GB of memory) with Ubuntu 24.04 LTS, in the `southamerica-west1` region (Santiago, Chile), with a static public IP. A firewall rule opens only TCP ports 5000 and 6000. Since nodes and clients start the connections, no ports had to be opened on the laptops' networks (Figure 6). The machine was prepared over SSH with a script that installs Java, downloads the data and compiles the project (Figure 7), and the server runs inside a `tmux` session so it keeps running after the SSH connection is closed (Figure 8).



Figure 6: The `oac-servidor` virtual machine in the Google Cloud console (Compute Engine).

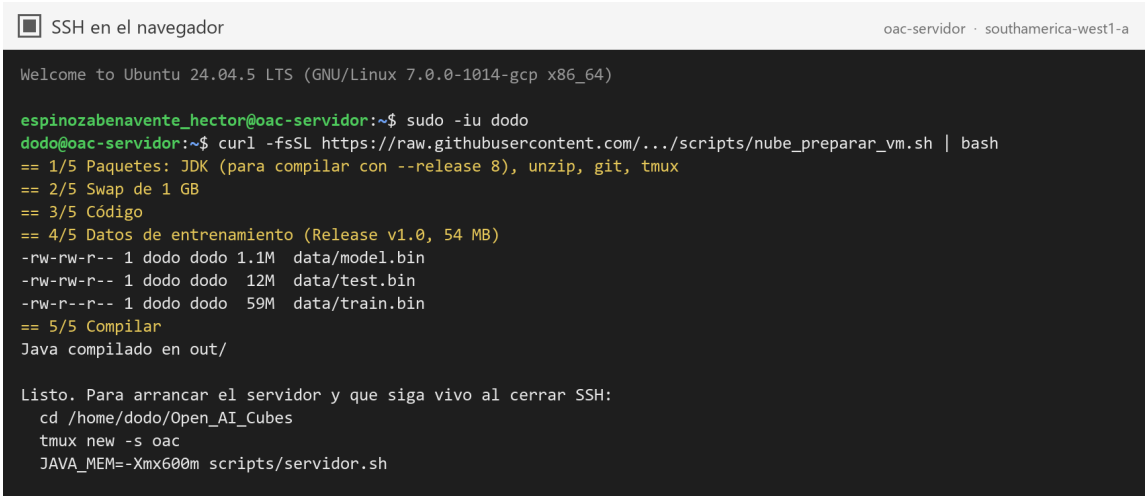


Figure 7: Server setup over SSH from the browser: Java installation, data download and compilation (trimmed output).

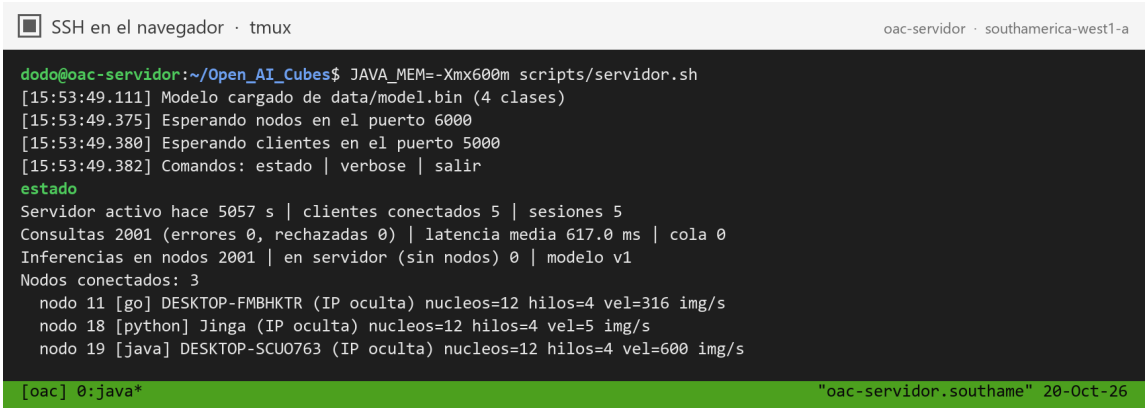


Figure 8: Central server running inside `tmux`, with the three team nodes connected (the laptops' IPs are hidden).

3.7 Experimental design

3.7.1 Scenarios and cases

The four scenarios of the assignment are measured for the three cases:

- **Sequential:** 1 process with 1 thread, on one machine.

- **Parallel:** 1 machine with 4 threads.
- **Distributed:** $M = 1, 2, 3$ laptops, with 1-thread nodes.
- **Parallel distributed:** $M = 1, 2, 3$ laptops, with 4-thread nodes.

The workload is the full CNN computation (*forward* and *backward*) over 512 images, with batch size 64 and the same initial weights in every run. As a result, the loss is identical in every case (1.3131), which confirms that all of them compute the same thing.

Parameter	Value
Images per run / batch	512 / 64 (1 epoch, SGD with $\eta = 0.01$ and momentum 0.9)
Repetitions	3 locally and 2 distributed; the median is reported
Local scenarios	Laptop 1, with the process pinned to performance (P) cores
Distributed scenarios	Server on Google Cloud and the 3 laptops over the internet
Case 3 when distributed	Mixed cluster: Java, Go and Python nodes, one on each laptop

Table 7: Parameters kept constant across the measurements.

3.7.2 Load test

The load generator simulates $N = 10, 50, 100, 200, 400$ and 800 concurrent clients, each with its own connection and session. Each client sends 20 single-image queries and waits a random *delay* of 0 to 200 ms between one query and the next. The test used the team configuration: the server on Google Cloud and the three nodes (Java, Go and Python) with 4 threads. In addition, the chat includes a load test button with the same *delay*, so that several real users can start it at the same time (Figure 13).

3.7.3 Metrics and instrumentation

For each distributed batch, the server separates the **computation** time (how long the slowest node takes) from the **communication and aggregation** time (sending weights and images, waiting and adding gradients). For each client level, the load generator records the responses per second and the mean and percentile latency (p50, p95 and p99). All results are stored in CSV files, from which the charts are generated.

4 Results and discussion

4.1 Time matrix (12 cases)

Scenario	Case 1: Java	Case 2: Python	Case 3: Go	Case 3: mixed
Sequential (1 thread)	1.99 s	20.37 s	2.07 s	–
Parallel (4 threads)	0.77 s	7.20 s	0.60 s	–
Distributed (3 laptops, 1 thread)	27.95 s	56.34 s	31.58 s	34.20 s
Parallel distributed (3 laptops, 4 threads)	21.77 s	28.81 s	21.95 s	23.78 s

Table 8: Total time to process 512 images (median). The mixed cluster does not exist on a single machine.

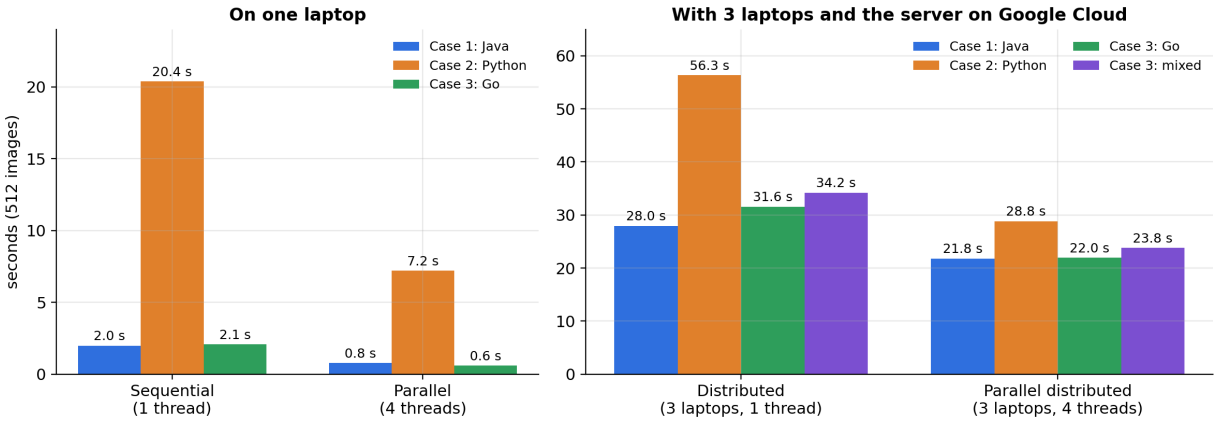


Figure 9: Time matrix: on one laptop (left) and with 3 laptops and the server on Google Cloud (right).

On a single machine, parallelism with 4 threads speeds up Java by 2.6×, Python by 2.8× and Go by 3.5×. Python is about ten times slower than Java and Go, because it does not use compiled numerical libraries. In the distributed scenarios the total time is higher than locally, despite using more machines: the network explains the difference, as the next section shows.

4.2 Scalability: speedup and communication cost

	Computation with 1 thread per node				4 threads
Case	**1 laptop**	**2 laptops**	**3 laptops**	**Speedup**	**3 laptops**
Java	5.29 s	2.51 s	1.84 s	2.9×	1.02 s (5.2×)
Python	57.23 s	30.04 s	19.14 s	3.0×	9.92 s (5.8×)
Go	5.20 s	3.38 s	2.15 s	2.4×	0.76 s (6.8×)

Table 9: Distributed computation time (without communication). Speedup is measured against 1 laptop with 1 thread.

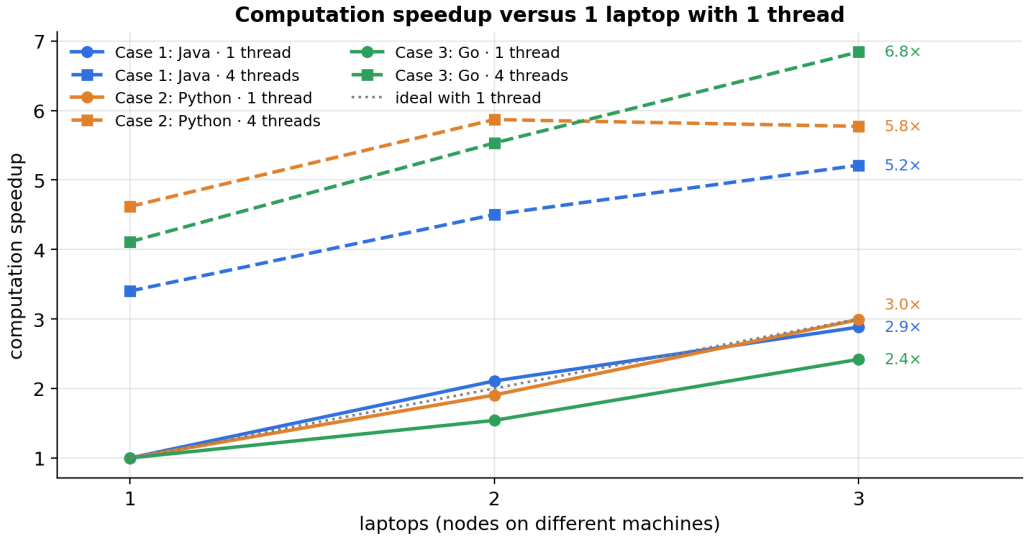


Figure 10: Computation speedup by number of laptops, with 1-thread nodes (solid line) and 4-thread nodes (dashed line).

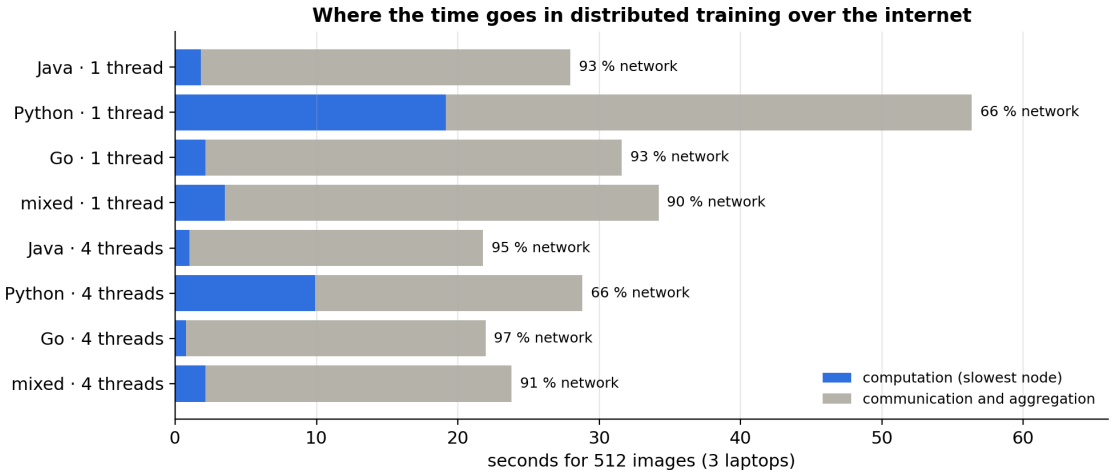


Figure 11: Computation time and communication and aggregation time with 3 laptops, by case and number of threads.

Computation scales almost linearly: with 3 laptops it is about 3 times faster, and with 4 threads per node, between 5 and 7 times faster (Figure 10). However, over the internet communication takes between 66 % and 97 % of the time (Figure 11). For each batch, the server sends the weights (about 1 MB) to each laptop and receives 1 MB of gradients back, through the upload link of a home network. Python is a partial exception: since it computes more slowly, the network weighs less in its total time. The mixed cluster is as fast as its slowest node in each batch, so its computation depends on the Python node, even though the proportional split gives it fewer images.

4.3 Load test

Clients	Queries	Responses/s	Mean latency	p95	Errors
10	200	32	135 ms	245 ms	0
50	1 000	162	159 ms	274 ms	0
100	2 000	312	178 ms	303 ms	0
200	4 000	572	203 ms	343 ms	0
400	8 000	704	420 ms	564 ms	0
800	16 000	753	900 ms	1 037 ms	0

Table 10: Load test with the server on Google Cloud and the three team nodes.

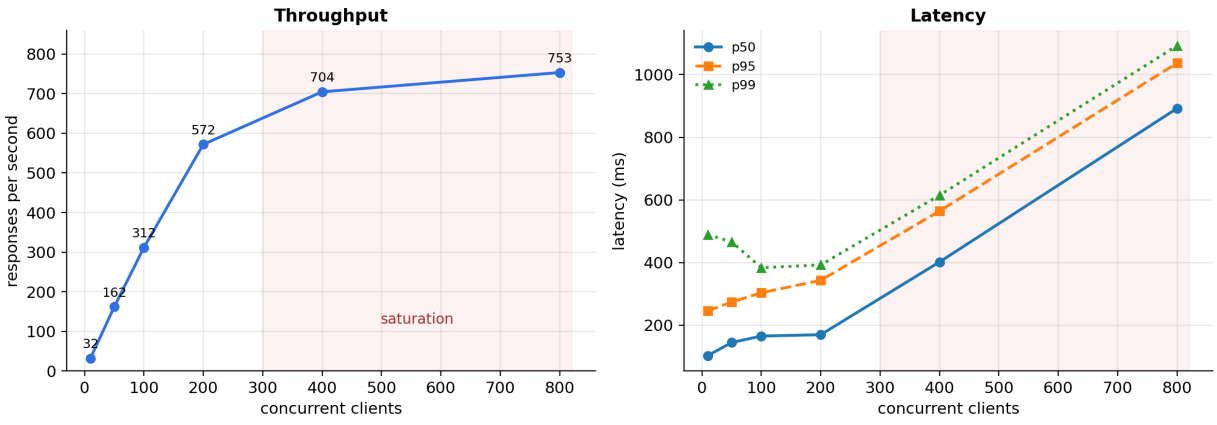


Figure 12: Throughput (left) and p50, p95 and p99 latency (right) versus the number of clients. The shaded area marks saturation.

All 31 200 queries were answered without errors. Up to 200 clients, throughput grows almost linearly and latency barely changes: the system has spare capacity. From 400 clients on, throughput levels off at about 700–750 responses per second, while latency doubles and then quadruples: that is the saturation point, where queries start waiting in the server queue. The limit is set by the cloud server, a small machine with 2 shared vCPUs.

With the chat button, a client sending 300 queries got a mean latency of 148 ms; when the three team members started it at the same time, latency was similar (119 ms). With three real clients the system receives about 14 queries per second, around 2 % of its capacity, so latency depends on the network (each query crosses the internet twice) and not on the load. Driving the system to saturation takes hundreds of clients, as in Table 10.



Figure 13: Load test button in the chat: mean latency, responses per second and queries solved by each node.

4.4 General discussion

The results show that parallelization works at both levels: inside each node, the micro chunks reduce computation time with threads, and across nodes the work is split with a computation speedup close to ideal. The bottleneck of the distributed system is the network: with nodes in different homes, the cost of sending weights and gradients in every batch exceeds the cost of the computation. This is the parallelization and aggregation cost studied in the course: to take advantage of distribution, a larger batch (fewer synchronizations per image) or a local network between the nodes is advisable.

The limits of the measurements should also be considered: two repetitions were made per distributed case, over a variable home network, and with laptops with different processors. Therefore, the figures describe the behavior of the complete system rather than a *microbenchmark* under identical conditions.

5 Conclusions

- A CNN with convolution, *pooling* and multilayer perceptron micro chunks was implemented in Java, Go and Python using only the standard library of each language. The three languages compute the same gradients, which makes it possible to combine them in a single cluster.

- The system integrates image processing, a *backend* with sessions and history and a chat *frontend*, with a custom binary protocol over TCP. It was deployed with the server on Google Cloud and three physical laptops as nodes, on different networks.
- Local parallelism speeds up between 2.6 and 3.5 times with 4 threads, and distributed computation scales about 3 times with 3 laptops. However, over the internet communication takes between 66 % and 97 % of the total time.
- The system answered 31 200 queries without errors. It saturates at about 750 responses per second, when latency goes from about 135 ms to about 900 ms.
- Fault tolerance worked under real conditions: the *heartbeat* detected unresponsive nodes and their work was reassigned without interrupting training.

6 Source code appendix

The complete source code of the project is included in the ZIP file submitted with this report. The main files are:

- `java/src/cubes/cnn/`: CNN, micro chunks and `ChunkExecutor`.
- `java/src/cubes/server/`: central server, inference queue, distributed training and node registry.
- `java/src/cubes/client/`: chat, load generator and admin console.
- `java/src/cubes/node/JavaNode.java`, `go/` and `python/node.py`: compute nodes in the three languages.
- `scripts/`: compilation, execution and measurements (`.bat` and `.sh`).

7 Documentation appendix

Requirements

- Java 8 or later (JDK to compile; the JRE is enough for the chat).
- Python 3.8 or later, for the Python node.
- Go 1.18 or later, only to compile the Go node.

Compilation

```
scripts\compilar.bat
```

Execution

On the server machine (on Google Cloud, `scripts/servidor.sh` is used):

```
scripts\servidor.bat
```

On each laptop, the node of its language, with the server IP and the number of threads:

```
scripts\nodo_java.bat   <server_ip> 4
scripts\nodo_go.bat     <server_ip> 4
scripts\nodo_python.bat <server_ip> 4
```

The chat, from any machine:

```
scripts\cliente.bat <server_ip>
```

Measurements

```
scripts\bench_local.bat 512
scripts\bench_distribuido.bat <server_ip> distribuido "1 2 3" 512
scripts\bench_distribuido.bat <server_ip> paralelo_distribuido "1 2 3" 512
scripts\carga.bat <server_ip>
scripts\graficas.bat
```

The results are stored as CSV files in `results/`, and the charts as PNG files in the same folder.

References

[1] CC4P1 – Programación Concurrente y Distribuida. *Examen Parcial 2026-II: Open AI Cubes*. Universidad Nacional de Ingeniería.

[2] CC4P1 course material. *Programación en red: Sockets I (04-SocketsA_ v10)*.

[3] CC4P1 course material. *Desafío de los Sistemas Distribuidos (Distribuidos)*.

[4] CC4P1 course material. *Sincronización de Hilos (scdHilos02)*.

[5] CC4P1 course material. *Monitores, condición de carrera y thread-safe (Hilos_ v12)*.

[6] CC4P1 course material. *Conceptos de Threads en Java (ThreadsA_ v07)*.

[7] CC4P1 course material. *Desempeño y optimización para la latencia (Desempeño251_ v03)*.

[8] A. Krizhevsky. *Learning Multiple Layers of Features from Tiny Images*. University of Toronto, 2009.

[9] Google Cloud. *Compute Engine documentation*. `https://cloud.google.com/compute/docs`.